

**SCHOOL-LEVEL SCIENCE AND ENGINEERING FAIR FOR SECONDARY
SCHOOL STUDENTS, SCHOOL YEAR 2026 – 2027**

REPORT ON THE RESEARCH'S FINDINGS

Project title:

**MEMOLANG: RESEARCH AND DEVELOPMENT
OF A BUFFER-RELEASE MECHANISM FOR AI
DEVELOPMENT**

***Category:* Systems Software**

Hanoi, September 2026

TABLE OF CONTENTS

PART I:	Question	4
I.1.	Algorithm	6
PART II:	Design and methodology	6
II.1.	Destructive-Read Semantics	7
II.2.	Compiler and Runtime Architecture	7
II.3.	Syntax Design	7
II.4.	The Free-on-Consume Scheme and cuBLAS runtime	8
II.5.	Pipeline	9
PART III:	Process, creation and testing	10
III.1.	Process	10
III.2.	Testing	10
III.3.	Benchmarks	10
III.4.	Expected Relationship Between Language and MatMul Per-	
	formance	11
III.5.	Evidence and Reproducibility	12
PART IV:	Conclusion	12
IV.1.	Progress	12
IV.2.	Future work	12
IV.3.	Meaning	12

LIST OF FIGURES

Figure 1.	Compilation pipeline of the language.	9
-----------	---	---

LIST OF TABLES

Table 1.	Scheme implementation choices in the design.	8
Table 2.	Main Source Files	10
Table 3.	Correctness Test Results	10

LIST OF ABBREVIATIONS

Abbreviation	Full name	Description
AI	Artificial Intelligence	Computer systems that perform tasks normally requiring human intelligence
AOT	Ahead-Of-Time (compilation)	Compiling code to machine code before the program runs
AST	Abstract Syntax Tree	Tree representation of the structure of source code
cuBLAS	CUDA Basic Linear Algebra Subprograms	NVIDIA's GPU-accelerated linear algebra library
CUDA	Compute Unified Device Architecture	NVIDIA's parallel computing platform for GPUs
CUTLASS	CUDA Templates for Linear Algebra Subroutines	NVIDIA's C++ template library for GPU linear algebra
DRAM	Dynamic Random-Access Memory	Main memory that must be periodically refreshed
FFI	Foreign Function Interface	Mechanism for calling functions written in another language
FP32	32-bit Floating Point	Single-precision floating-point number format
GEMM	General Matrix Multiplication	Standard routine for multiplying two matrices
GPU	Graphics Processing Unit	Processor specialized for highly parallel computation
IR	Intermediate Representation	Code form used inside a compiler between source and machine code
JIT	Just-In-Time (compilation)	Compiling code to machine code while the program runs
LLM	Large Language Model	AI model trained on large amounts of text
LLVM	LLVM Compiler Infrastructure (originally "Low Level Virtual Machine")	Toolkit for building compilers and code generators
MatMul	Matrix Multiplication	Multiplication of two matrices
REPL	Read-Eval-Print Loop	Interactive shell that evaluates input line by line

Report on the Research's Findings

Memo-Lang: Destructive-Read Semantics on GPU Matrix Workloads

Independent Interpreter Project

September 2026, Hanoi

September 28, 2026

Abstract

Sự phát triển của AI trong thời gian gần đây đã dẫn đến sự bùng nổ của nhiều thuật toán máy học và mô hình học sâu. Do vậy, nhu cầu cho năng lực tính toán để học trên những bộ dữ liệu khổng lồ cũng từ đó tăng.

Hiện nay, phép toán được sử dụng rộng rãi cho thuật toán máy học là GEMM (General Matrix Multiplication - Phép nhân ma trận tổng quát). Trong quá trình học, mỗi phép nhân ma trận sẽ tạo ra một ma trận khác và ma trận mới sẽ được nối với những phép nhân ma trận khác. Để xử lý nhiều thao tác, tác vụ và quá trình khác nhau, những mô hình máy học cần thực hiện hàng triệu phép tính ma trận. Do những phép tính ma trận này so với các phép tính thông thường khác có tính đặc thù về thời gian rất cao, vì vậy những phép tính tiếp theo dần bị dồn lên bộ nhớ đệm và các bước trung gian cho đến khi những chương trình quản lý bộ nhớ hoặc quy tắc của máy tính giải phóng bộ nhớ.

Việc lặp lại quá trình này đối với mô hình máy học trở thành một vấn đề lớn trong việc hoạt động dưới một khối lượng công việc rất lớn, để lại hậu quả lớn cho hiệu suất và độ trễ của mô hình trên thiết bị và tài nguyên phần cứng nó đang sử dụng.

Bởi vậy, phần mềm đồng thời góp phần tối ưu hóa nhân của GEMM và tăng tuổi thọ của phần cứng được sử dụng, đảm bảo bộ đệm, dữ liệu đệm và con trỏ được giải phóng sớm nhất có thể và giảm phí tổn từ kết quả của việc dồn bộ nhớ đệm.

PART I: Question

Our main goal is to directly link the precompiled *cuBLAS runtime library* for a specific application, evaluated with a specific methodology, under an enforcement model resembling a *strictly linear type system*. Other languages have pursued related consumption-tracking semantics: Rust's ownership system [6] enforces a comparable use-once discipline through its affine type system, and Futhark [7] is a purely functional array language that uses uniqueness types to support safe in-place updates. Futhark, however, generates its own GPU kernels from source rather than integrating with external, independently maintained high-performance libraries such as cuBLAS, a limitation that has been noted

as one of the practical hurdles of the language. The language instead separates compile-time consumption analysis from kernel execution.

$$T_1 = AB, \quad T_2 = T_1C, \quad T_3 = T_2D, \quad (1)$$

In implementation, numerous intermediate buffers are live at the same time. These buffers are best have their storage be erased and reused after their respective operations has run out or lifetime of the object has ran out.

Memory management benefits can be expected from a destructive-read semantics language. While arithmetic operations such as the Matrix Multiplication routine can be delegated to an optimized CUTLASS/cuBLAS kernel to circumvent [7] Futhark’s limitations. Implementation is expected to have no effect on the computational performance compared to implementation on other languages. End-end improvements, however, is expected to be present due to the optimization of the language including:

- Reduced temporary allocation
- Reduced buffer lifetime
- Reduced GPU memory pressure

In theory, this implementation’s effectiveness scales with the workloads, as immediate tensors or smaller matrix operations do not observe a significant impact from memory management compared to larger chained workloads.

However, inherent overheads or overestimation of memory overhead or memory optimization’s practicality is important to consider. Therefore, The project investigates the following hypotheses:

- Destructive-read semantics can reduce peak memory usage for workloads containing short-lived intermediate matrices.
- Destructive reads can reduce the number or cost of temporary-buffer allocations when consumed buffers can safely be reused.
- For workloads in which memory management represents a significant portion of total execution time, buffer reuse may improve end-to-end performance.

This project was inspired by research into destructive-read memory systems, the Sere programming language [5], the Clean programming language [4], and optimized GPU matrix-multiplication libraries.

Then, Potential real-world benefits include:

- Improved performance for matrix multiplication operations, supporting faster training and inference for machine learning models.
- More effective compiler-driven reuse of intermediate tensor buffers.
- Reduced run-time buffer and cache allocation overhead.

I.1. Algorithm

Matrix multiplication can be written as

$$C_{i,j} = \sum_{k=0}^{N-1} A_{i,k} B_{k,j} \quad (2)$$

Each value in the output matrix C is produced by multiplying one row of A with one column of B and summing the products. Efficient implementations must manage data movement, memory reuse, and, in a destructive-read scheme, the lifetime of operand buffers once they are consumed.

Destructive-read is a memory read operation that inherently erases the value it reads: data is destroyed at the same time it is accessed. If a value must be preserved, a write-back operation must immediately restore it, or an external buffer/cache must hold a copy.

Related work on linear type systems takes a similar approach at the language level, requiring that a variable be used exactly once.

Destructive-read has, historically, been more common in hardware and circuit design than in software. Dybdahl et al. [1] studied destructive-read behavior in embedded DRAM and found that removing the write-back buffer and instead relying on the cache by reading data into the cache without immediately restoring it to DRAM, and writing it back only when the cache line is replaced, offered measurable power and performance benefits.

While that result is specific to a hardware memory system, it motivates the broader question this project investigates: whether a comparable consume-on-read discipline, applied at the software/interpreter level for GEMM-heavy workloads, can yield similar results?

PART II: Design and methodology

The goal of the language is to determine whether language-level destructive-read semantics can provide measurable memory-management benefits for matrix-oriented GPU programs while retaining the performance of optimized matrix multiplication kernels.

The project will be implemented and evaluated through the following goals:

- Define language’s core Python-like syntax and destructive-read semantics.
- Implement a lexer, parser, AST, and LLVM-based code-generation pipeline.
- Generate LLVM IR for arithmetic and matrix operations.
- Integrate optimized GPU matrix multiplication through CuBLAS.
- Implement explicit ownership and consumption tracking for matrix operands.
- Determine when consumed matrix buffers can safely be reused.
- Verify generated results against an independent numerical reference.

- Measure peak GPU memory usage and temporary-buffer allocation behavior.
- Measure both GPU kernel execution time and end-to-end program runtime.
- Compare destructive-read execution against an equivalent non-destructive implementation.
- Compare language against optimized GPU-library baselines where appropriate.
- Preserve benchmark logs, correctness results, compiler output, and build artifacts to make the experimental results reproducible.

The evaluation will not assume that destructive reads provide a performance improvement. A reduction in peak memory usage without a reduction in runtime will be treated as a meaningful result, while an observed runtime improvement will be analyzed to determine whether it originates from reduced memory-management overhead rather than from changes to the underlying matrix multiplication kernel.

II.1. Destructive-Read Semantics

A destructive read treats a value as consumed when an operation has exclusive use of that value. After consumption, the original value is no longer available to subsequent operations unless the programmer explicitly creates a copy or otherwise preserves it.

For matrix operations, this provides an opportunity for storage reuse. For example,

```

1   C = A @ B
2   D = C @ W
3
```

may permit the storage associated with *A* or *C* to become eligible for reuse once the compiler establishes that the corresponding value has no remaining consumers.

The purpose of this mechanism is not to alter the mathematical computation performed by matrix multiplication. Instead, it changes the lifetime of buffers associated with the computation. If a buffer can be reused without copying its contents, the implementation may reduce peak memory consumption and allocation activity.

This optimization is inherently workload-dependent. A matrix with multiple future consumers cannot safely be destroyed after its first use. In such cases, the implementation may require a copy, borrow, or other mechanism to preserve the value. Consequently, destructive-read semantics can introduce a tradeoff between memory efficiency and the flexibility to reuse values.

The language therefore treats destructive reads as an explicit ownership mechanism rather than assuming that consuming every operand will necessarily improve performance.

II.2. Compiler and Runtime Architecture

II.3. Syntax Design

We've decided on a Python-like syntax convention for the programming language:

- Comments using hashtag.
- Primitive Types: Integer, Float, Boolean and String.
- Variable declarations .
- Functions using 'def'.
- extern for runtime/FFI(cross-language) linkage.
- operator precedence.
- control flow.

However, due to implementation practicality and further design decision, we've decided to make adjustments to the core Python syntax:

- special precedence of clone() for cloning memory
- usage of braces instead of indentation for function scope

II.4. The Free-on-Consume Scheme and cuBLAS runtime

The scheme referenced in [1] will be used with a modified design philosophy to work at a software-level.

Table 1. Scheme implementation choices in the design.

Scheme	Implementation
Data is first read from DRAM into the cache.	No difference.
Data is written back to DRAM when the cache line is replaced, regardless of whether data was modified.	Operands are freed unconditionally under the assumption that the cache is always modified.
Reuses existing cache architecture.	Uses scope-exit allocation for cache management.
Hides write-back latency using multi-bank overlap (bank interleaving).	Hides memory free overhead behind GPU kernel launches.

for cuBLAS pre-compile linking:

- cuBLAS should be dynamically linked.
- Driver should link and ship with compiled binaries from cuBLAS.
- For convenience, distributed as a packaged file.

II.5. Pipeline

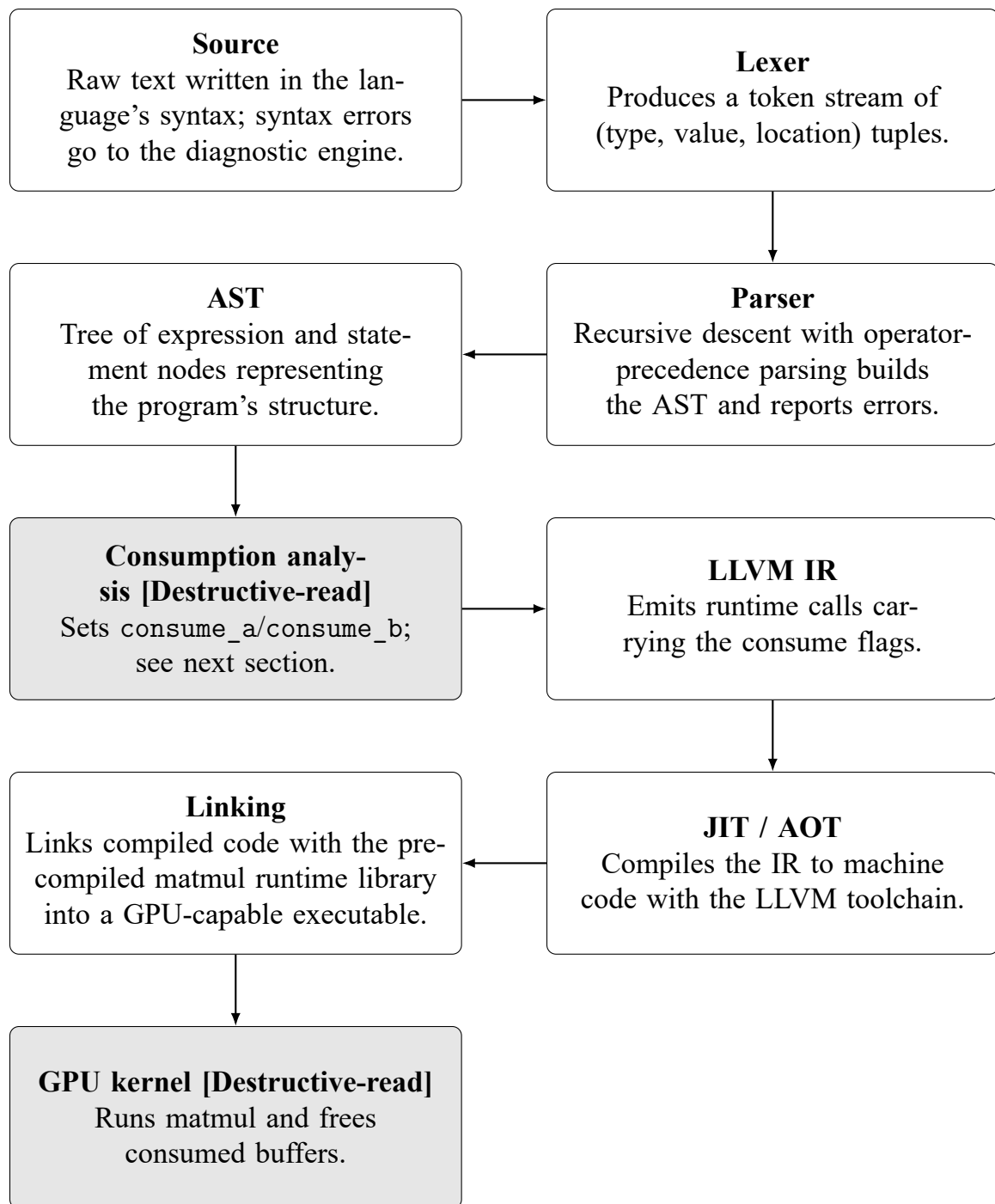


Figure 1. Compilation pipeline of the language.

The compiler front-end is written in C++, each stage is first designed, then coded, then debugged independantly to look for errors and the assembled modularly.

Table 2. Main Source Files

File	Purpose
Lexer.h/.cpp	Tokenizes language source
Parser.h/.cpp	Builds the AST
ExprAST.h	AST node definitions
CodeGen.cpp	LLVM IR generation
CuBLASRuntime.cpp	cuBLAS MatMul bindings
main.cpp	Driver / REPL

PART III: Process, creation and testing

III.1. Process

III.2. Testing

Table 3. Correctness Test Results

Test Case	Result
$N \times N$ MatMul vs. reference (conventional)	TBD
Destructive-read re-use error detection (runtime)	TBD
Edge cases (empty, 1x1, non-square)	TBD

$N \times N$ MatMul vs. reference (conventional): Evaluating a chain of matrix multiplication operations to stress-test for evaluating end-to-end improvements. Destructive-read re-use errordetection: Evaluates at run and compile time to sort for errors Edge cases: Evaluate edgcases to stress test compatability and thoroughness of the algorithm.

III.3. Benchmarks

The benchmark evaluation is designed to distinguish improvements in memory management from improvements in matrix multiplication itself.

Three primary implementations will be compared:

1. An optimized GPU-library implementation used as a kernel-performance reference.
2. language using conventional non-destructive value semantics.
3. language using destructive-read semantics.

Where appropriate, a simple C++ or CUDA implementation will also be included as a baseline.

The most important comparison is between the two languages configurations. Both configurations perform the same mathematical operations and use the same underlying GPU matrix multiplication implementation. This comparison isolates the effect of

destructive-read semantics more effectively than a comparison against an unrelated implementation.

The benchmark suite will include isolated matrix multiplications as well as chains of matrix operations. Matrix dimensions will include small, medium, large, and non-square matrices. This allows the evaluation to determine whether the effect of destructive reads depends on workload size.

The following measurements will be collected:

- GPU kernel execution time.
- End-to-end execution time.
- Peak GPU memory consumption.
- Number of temporary-buffer allocations.
- Number of buffer reuse events.
- Host-to-device and device-to-host transfer time where applicable.
- Compiler and runtime overhead.
- Numerical error relative to a reference implementation.
- GPU power draw.

GPU measurements will use multiple repetitions and appropriate warm-up iterations. CUDA [] initialization and compilation overhead will be separated from steady-state execution measurements where possible.

The benchmark will report both absolute measurements and relative changes. No performance improvement will be attributed to destructive-read semantics unless the corresponding measurements support that interpretation.

III.4. Expected Relationship Between Language and MatMul Performance

The expected execution time can be approximately decomposed as

$$T_{\text{total}} = T_{\text{language}} + T_{\text{memory}} + T_{\text{launch}} + T_{\text{kernel}}. \quad (3)$$

Destructive-read semantics primarily target T_{memory} . They are not expected to substantially reduce T_{kernel} when the same optimized GPU kernel is used.

For a large matrix multiplication, T_{kernel} may dominate the total execution time, resulting in little observable end-to-end improvement. For workloads containing many intermediate tensors or small matrix operations, however, T_{memory} and other runtime costs may become more significant.

Deducing the language’s effectiveness primarily relies on measuring T_{memory} .

III.5. Evidence and Reproducibility

Example build:

```
1  def basicadd(int i, int k) {  
2      return i + k;  
3  }  
4  # Basic compilation of source code, addition only. No showcase of  
   GEMMs or cuBLAS integration.  
5
```

Listing 1. Example compilation

PART IV: Conclusion

IV.1. Progress

The project has completed a portion of the planned work. This includes the lexer, the parser, the AST, and a basic driver implementation. Design for the remaining pipeline has been finalized, but implementation of the features further down the pipeline has not yet begun due to time constraints.

IV.2. Future work

Features further down the pipeline, including integration of the precompiled matmul runtime library, remain to be implemented. The currently planned features are as follows:

- Vector and matrix types
- A diagnostic driver and diagnostic engine
- Static consumption analysis (destructive-read checking) and handling of GPU kernel buffers and intermediates
- Linking against the precompiled matmul runtime to produce an executable capable of launching the GPU kernel
- Separation of code generation from the AST for cleaner lowering
- Optimization passes
- A macro system (link declarations, matmul-chain pattern abstraction)

IV.3. Meaning

This project serves both as an experiment to explore potential improvements to GEMM operations for machine learning and AI algorithms, complementing the already highly-optimized cuBLAS [2] library, and as a response to a broader problem in the field. Current model training consumes massive amounts of memory, and demand from AI infrastructure has outpaced global memory chip supply, contributing to a sustained rise

in DRAM and NAND prices that has placed financial pressure on IT firms, scientific research institutions, and consumers alike [12]. Computing algorithms for GEMM operations are already optimized to *near theoretical limits* in raw throughput; this research instead targets memory efficiency, using a novel ownership system to reduce the memory footprint required per workload, as one contribution toward easing the demand-side pressure driving the current memory shortage.

References

- [1] Haakon Dybdahl, Per Gunnar Kjeldsberg, Marius Grannæs, and Lasse Natvig, “Destructive-read in embedded DRAM, impact on power consumption,” *Journal of Embedded Computing*, vol. 2, no. 1, pp. 83–96, 2006.
- [2] NVIDIA, “cuBLAS Library.” <https://developer.nvidia.com/cublas>
- [3] LLVM Project, “The LLVM Compiler Infrastructure.” <https://llvm.org>
- [4] Clean Language Team, “The Clean Programming Language.” <https://clean.cs.ru.nl>
- [5] Sere Language Project, “Sere Programming Language.” <https://clean-lang.org>
- [6] N. D. Matsakis and F. S. Klock, “The Rust language,” in *Proc. ACM SIGAda Ada Letters*, 2014. <https://www.rust-lang.org>
- [7] T. Henriksen, N. G. W. Serup, M. Elsmann, F. Henglein, and C. E. Oancea, “Futhark: purely functional GPU-programming with nested parallelism and in-place array updates,” in *Proc. 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017. <https://futhark-lang.org>
- [8] Independent Interpreter Project, “MemoLang: source and benchmark evidence,” project repository, September 2026.
- [9] NVIDIA, “CUTLASS Library.” <https://github.com/nvidia/cutlass>
- [10] NVIDIA, “Mixed-precision training.” <https://docs.nvidia.com/deeplearning/performance/mixed-precision-training/index.html>
- [11] NVIDIA, “Accuracy Considerations” <https://docs.nvidia.com/deeplearning/tensorrt/latest/inference-library/accuracy-considerations.html>,
- [12] TrendForce, “Memory Price Outlook for 1Q26 Sharply Upgraded,” <https://www.trendforce.com/presscenter/news/20260202-12911.html>,
- [13] NVIDIA, “CUDA ” <https://developer.nvidia.com/cuda/toolkit>